

PRODUCT REQUIREMENTS DOCUMENT

Flash-Style AI Shopping Discovery Platform

A build-ready PRD covering the product, the technology stack, the API surface, and the end-to-end delivery process for an AI-agent-curated shopping and recommendation product.

Reference product	flash.co (Shop the top 1%. Run by AI)
Document type	Product Requirements Document
Version	1.0
Date	October 5, 2026
Status	Draft for review
Classification	Internal / Confidential

Prepared from direct analysis of flash.co public pages and standard industry architecture. Not affiliated with or endorsed by Flash.co.

Table of Contents

1. Executive Summary
2. Background & Market Context
3. Reference Product Analysis: flash.co
4. Goals, Non-Goals & Success Metrics
5. Target Users & Personas
6. Functional Requirements
7. Non-Functional Requirements
8. System Architecture
9. Technology Stack
10. Data Model
11. API Specification
12. AI Agent Research & Ranking Pipeline
13. Complete Project Flow - Zero to Deployment
14. Build Roadmap, Milestones & Delivery Process
15. Knowledge, Concepts & DSA Required
16. API & Services Catalog (Free vs Paid)
17. Project Requirements (Hardware, Software, Skills, Budget)
18. Team, Effort & Cost Model
19. Risks & Mitigations
20. Compliance, Legal & Trust
21. Appendices (Observed fingerprints, sample payloads, glossary)

HOW TO READ THIS DOCUMENT

Sections 1-7 define WHAT to build and for whom. Sections 8-12 define HOW (architecture, stack, data, API, AI pipeline). Sections 13-17 define the DELIVERY path: the complete zero-to-deployment flow, the roadmap, the full set of knowledge/concepts and DSA required, an API/service catalog with free-vs-paid status, and the hardware/software/skill/budget requirements. Sections 18-21 cover team/cost, risks, compliance and raw evidence. If you are building this as an academic/DSA project, read Sections 13, 15 and 17 first.

1. Executive Summary

This PRD specifies an AI-agent-curated shopping discovery platform modeled on flash.co. The product's core idea is a reversal of traditional e-commerce search: instead of listing every product, autonomous AI agents continuously research the entire market, verify claims against independent evidence, score and rank products, and publish only a small, defensible shortlist ("the top 1%") with the best available price across many stores.

The platform is not a marketplace and holds no inventory. It is a discovery, research, ranking and referral layer. Revenue is earned through affiliate commissions on outbound clicks to merchants such as Amazon, Flipkart, Myntra and Nykaa. Trust is the primary asset: rankings are explicitly non-sponsored and every claim is traceable to a named source.

1.1 Product in one paragraph

Users browse curated categories, read evidence-backed ranked lists, open a product to see its Flash Score, pros/cons, specs, and live prices across stores, ask a natural-language question in a chat interface whose answers are grounded in the same agent research, and click through to buy at a partner store. A public "Agent Hub" streams the agents' live activity, making the research process itself a trust-building feature.

1.2 What is in scope for v1

1.3 What is explicitly out of scope for v1

- Holding inventory, payments to merchants, fulfilment, returns, or customer support for orders.
- A full seller onboarding / marketplace CMS.
- Native mobile apps (responsive web + PWA first).
- Internationalization beyond India; single currency (INR) and India-first merchants.

2. Background & Market Context

Online shopping discovery has two structural problems. First, choice overload: a single product category can contain thousands of SKUs, and marketplace search optimizes for what sells or what is promoted, not what is best for the buyer. Second, trust erosion: ratings and reviews are gamed by seeded reviews, paid placements and review farms, so aggregate star ratings are an unreliable signal.

Large language models can summarize the web, but their answers are non-deterministic (the same question can yield different picks), frequently stale (cached training data), and shallow (they read a sliver of available sources). The opportunity is to combine the reasoning of LLMs with a disciplined, always-running research pipeline that reads broadly, verifies independently, and produces a stable, auditable ranking.

2.1 Why now

- Frontier LLMs are now strong enough to extract structured attributes from unstructured reviews and video transcripts at low unit cost.
- Cheap, scalable web crawling, transcript and API access (YouTube, Reddit, shopping feeds) make broad evidence collection feasible.
- Consumers increasingly begin product research inside AI assistants, creating a new discovery surface to own.
- Affiliate economics in India are mature, with deep-link programs at Amazon, Flipkart, Myntra, Nykaa and others.

2.2 Competitive framing

Approach	What it gives	Where it fails
Marketplace search	Huge catalogue, transactional intent	Pay-to-rank; catalogue-only; reviews gamed
Review/aggregator sites	Volume of reviews	Low signal, SEO-driven, no live re-ranking
Generic LLM chat	Natural language answers	Stale, non-deterministic, shallow sourcing
Flash-style agent platform	Verified shortlist + live prices + provenance	Higher build/operational cost; evidence pipeline complexity

3. Reference Product Analysis: flash.co

The following is based on direct inspection of public flash.co pages (/home, /browse, /how-flash-works, /hub, /articles, /list/{id}, /item/{id}) and the client-side artifacts served to browsers.

3.1 Positioning & value proposition

Tagline: 'Not every product. Only the top 1%.' and 'Shop the top 1%. Run by AI.' The claim is that agents read everything several times over, validate every rating and review against Reddit, YouTube, experts and real buyers, discard seeded reviews, and assign one score per product. The promise is: buy right the first time, at the best price.

3.2 Core surfaces observed

Surface	Route pattern	Purpose
Home	/home	Hero, featured cards, freshly re-ranked lists, category entry points, best-scored carousels
Browse	/browse	Full category taxonomy (Beauty, Electronics, Health & Wellness, etc.)
Discovery	/discovery/{id}	Attribute/category landing that groups related lists
List	/list/{id}	Ranked list of products for a use-case, with research provenance
Item	/item/{id}	Product detail: score, verdict, research, specs, rankings, prices
Prices	/item/{id}/prices	Multi-store price comparison
Brand	/brand/{slug}	Brand profile and its ranked placements
Search	/search?query=	Keyword search entry
Ask Flash	/hub	Agent hub: live run log plus natural-language Q&A grounded in research
The Cut	/articles	Editorial articles derived from research (routines, how-to, think-twice, ingredients)
Redirect	/go?u=<encoded>	Outbound affiliate deep-link redirect with tracking
How it works	/how-flash-works	Explainability page describing the agent pipeline
Profile	/profile	User settings and history

3.3 Observed feature behaviours

- Each list shows evidence counts: e.g. '6,155 expert analysis, 4,436 YouTube, 11,019 reviews, 1,023 Reddit threads' and a ranked count such as 'TOP 1094'.
- Each product carries a Flash Score out of 100 (e.g. 94/100) plus a short verdict, 8+ findings, pros/cons and 'best for' tags.
- Items show 'Agent found it in 3 stores' and a lowest-price merchant, with offers and bank/coupon details.

- Rankings are contextual: the same product is #4 of 61 in one list, #5 of 64 in another, #166 of 833 in a broad list, with re-rank dates.
- The Agent Hub streams live events by type (Reviews, Rankings, Prices, Fact-check, Expert, YouTube, Reddit, Guides), with run numbers (e.g. RUN #4,182) and a 24h/72h/7d event histogram.
- The home page includes 'attribute clusters mined from product pros' that no existing list covers — demonstrating automatic list generation.
- Affiliate deep links embed tracking (Amazon tag=flashai-21, Flipkart affid=flashco).
- Editorial pages ('The Cut') state 'Nothing here is sponsored. Every claim names the source it came from.'

3.4 Observed technical fingerprints

These were identified from response headers and the client bundle, and inform the recommended stack in Section 9.

Layer	Observed signature	Implication
Frontend framework	Next.js App Router with Turbopack; React Server Components; /_next/static chunks	Modern SSR/RSC React app
UI system	MUI (MuiBox-root, MuiSvgIcon-root) + Emotion (data-emotion) + Tailwind utilities	Component library plus utility CSS
Typography	Inter Tight, Playfair Display, Instrument Serif, DM Sans, JetBrains Mono, Bangers, Lilita One	next/font, brand-led type system
Edge/CDN/WAF	server: cloudflare; cf-ray; cf-cache-status; NEL reporting	Cloudflare in front of origin
Media pipeline	i.flash.tech / img.flash.co transform URLs over s3://flash-creatives/... (resize, webp, quality)	On-the-fly image transforms backed by S3
Analytics	googletagmanager.com dns-prefetch/preconnect	GTM-based tag management
Affiliate	/go?u=<urlencoded> with Amazon/Flipkart affiliate params	Server-side redirect + attribution
Data sources	Google Shopping (gstatic) imagery + Amazon/Flipkart/Myntra/Nykaa/Sweetcare etc.	Aggregated multi-merchant catalogue
SEO	Canonical, OG/Twitter cards, per-list titles	Content/SEO-led acquisition

4. Goals, Non-Goals & Success Metrics

4.1 Product goals

- G1: Let a user go from a vague need to a confident purchase decision in under five minutes.
- G2: Publish rankings that are evidence-backed, reproducible and resistant to manipulation.
- G3: Keep prices, availability and rankings live (re-ranked continuously, not quarterly).
- G4: Make the research process transparent so users trust the recommendation.
- G5: Monetize through affiliate referral without compromising ranking integrity.

4.2 Non-goals

- N1: Becoming a marketplace or inventory holder in v1.
- N2: Selling rank placement or sponsored listings.
- N3: Covering every long-tail category on day one.

4.3 Success metrics (north star & KPIs)

Metric	Definition	v1 target
North star	Qualified outbound clicks per week	Baseline +30% QoQ
Decision confidence	% sessions with a click-through to a store	>= 18%
Research coverage	Median independent sources per ranked product	>= 40
Freshness	Median days since last re-rank for top lists	<= 14 days
Price accuracy	Price matched at redirect time	>= 97%
Ask Flash usefulness	Answers with >=1 citation and product link	>= 90%
Retention	4-week returning visitors	>= 25%
Integrity	Rank changes triggered by payment	0 (hard invariant)

5. Target Users & Personas

Persona	Context	Needs	Key features
The Overwhelmed Buyer	Wants a good product, hates comparing	Fast, trustworthy shortlist	Curated lists, top pick, Flash Score
The Optimizer	Researches heavily before buying	Evidence, comparisons, best price	Research tab, rankings, price compare
The Deal Seeker	Price-sensitive, timing-aware	Lowest live price, coupons	Price history, store offers, affiliate links
The Gift/Novice Shopper	Doesn't know the category	Plain-language guidance	Ask Flash chat, editorial routines
The Skeptic	Distrusts reviews/ads	Provenance and independence	Agent Hub, source citations, 'nothing sponsored'

Primary launch audience: urban India, 22-40, mobile-first, shopping across beauty/personal care, consumer electronics and health supplements.

6. Functional Requirements

Each requirement has an ID, a priority (P0 = must-have for v1, P1 = should-have, P2 = later) and acceptance criteria. Priorities are indicative for a 12-16 week v1.

6.1 Catalogue & taxonomy

1. FR-1 (P0) Category taxonomy. The system maintains a hierarchical taxonomy (Vertical > Department > Category > Attribute). Example path: Beauty / Skincare / Face Moisturizers. Acceptance: browse page renders all verticals with category tiles and 'view all' expansion.
2. FR-2 (P0) Canonical product records. Every product is deduplicated across merchants into one canonical entity with a stable item ID, brand, title, images, specs and GTIN/SKU aliases where available. Acceptance: the same SKU from Amazon and Flipkart resolves to one item page.
3. FR-3 (P0) Merchant/offer records. Each canonical product links to one or more offers (merchant, price, MRP, coupon, availability, deep link). Acceptance: item page shows the lowest of N stores and a compare view.
4. FR-4 (P1) Brand profiles. Brands aggregate their products and ranked placements, with badges such as 'Category owner' and 'Most ranked'.

6.2 Curated lists & attribute clusters

5. FR-5 (P0) Ranked lists. A list has a title, intro, category path, ranked items with rank, score, price and merchant, plus provenance counts (expert/YouTube/reviews/Reddit). Acceptance: list page matches the observed layout and supports pagination/virtualization to 1,000+ items.
6. FR-6 (P0) Re-ranking with dates. Each list displays 're-ranked <date>' and a link to 'see what ran'. Acceptance: re-rank events are logged and surfaced.
7. FR-7 (P1) Attribute clusters. The system automatically mines attribute clusters (e.g. 'Sunscreens with No White Cast', 'Supports muscle recovery whey protein') and creates new lists when no existing list covers the cut. Acceptance: home page shows at least one auto-generated cluster.
8. FR-8 (P1) Faceted list filters. Price and other attribute filters operate over a list. Acceptance: filtering preserves ranking order and updates result count.

6.3 Scoring, evidence & explainability

9. FR-9 (P0) Flash Score. Every product receives a 0-100 score derived from a documented, weighted rubric across attributes relevant to the list context. Acceptance: score is reproducible from stored evidence and rubric version.
10. FR-10 (P0) Findings & downsides. Each product page lists structured findings (pros) and downsides extracted from sources, each traceable to a source. Acceptance: at least one source reference per finding.
11. FR-11 (P0) Source validation. Ratings/reviews are validated against independent sources; seeded reviews are detected and discarded. Acceptance: a review-authenticity score is stored and reflected in scoring.
12. FR-12 (P1) Fact-check agent. A critic agent verifies rank/score consistency and flags products that lack attribute scoring. Acceptance: hub shows fact-check events and flags propagate to internal review queues.

6.4 Search & Ask Flash

- 13. FR-13 (P0) Keyword search. Search across products, lists, brands and articles with typo tolerance. Acceptance: top-10 results contain a relevant product/list for benchmark queries.
- 14. FR-14 (P0) Ask Flash (RAG chat). Natural-language Q&A grounded in the agent research corpus, streaming answers with citations and product cards. Acceptance: answers never invent products outside the corpus; each answer cites sources.
- 15. FR-15 (P1) Paste-a-link / reel analysis. Users can paste a product URL or Instagram reel and get an analysis mapped to the catalogue. Acceptance: valid inputs return a product/attribute match within 10s.
- 16. FR-16 (P1) Chat history. Authenticated users can revisit recent chats and saved searches.

6.5 Price comparison & monetization

- 17. FR-17 (P0) Multi-store price comparison. Show all known offers for an item, lowest first, with coupons and bank offers. Acceptance: compare page lists ≥ 1 offer per available merchant with freshness timestamp.
- 18. FR-18 (P0) Affiliate redirect. Outbound clicks route through a tracked redirect that appends the correct affiliate parameters per merchant and logs the click. Acceptance: redirect returns 302 with correct params; click is attributable to user/session/list/item.
- 19. FR-19 (P0) Affiliate disclosure. Every outbound/affiliate context displays a clear disclosure. Acceptance: disclosure present on item, list and redirect interstitial as required.

6.6 Live agents & editorial

- 20. FR-20 (P0) Agent Hub live feed. Stream agent events (type, message, timestamp, related item) over SSE/WebSocket with filters and a time histogram. Acceptance: events appear within 5s of generation; filters work.
- 21. FR-21 (P1) Run log. Show numbered pipeline runs (e.g. RUN #4,182) and per-run details. Acceptance: run detail lists stages and outputs.
- 22. FR-22 (P1) The Cut editorial. Publish research-backed articles with vertical/section taxonomy, citations and internal product links. Acceptance: article pages are indexable and link to relevant lists/products.

6.7 Accounts & personalization

- 23. FR-23 (P0) Authentication. OTP/email or OAuth sign-in; guest browsing allowed. Acceptance: sign-in persists sessions; profile page accessible.
- 24. FR-24 (P1) Saved items & chats. Users can save products/lists and revisit chats. Acceptance: saved state persists server-side.
- 25. FR-25 (P2) Personalized ranking. Re-rank recommendations using on-site behaviour with explicit, opt-outable signals. Acceptance: measurable CTR lift in A/B test without violating ranking-integrity invariant.

7. Non-Functional Requirements

Category	Requirement	Target
Performance	LCP on mobile 4G for list/item pages	<= 2.5s
Performance	Ask Flash first token	<= 1.5s (streaming)
Scalability	Concurrent users v1	50k MAU, 5k peak concurrent
Availability	Public site uptime	99.9% monthly
Freshness	Price re-crawl cadence (top items)	<= 6 hours
Freshness	Ranking re-run cadence per list	<= 14 days (hot lists weekly)
Data quality	Price accuracy at redirect	>= 97%
Security	Secrets, PII, least privilege	OWASP ASVS L2; no PII in logs
Privacy	Consent + data subject rights (DPDP)	Consent gate; export/delete
SEO	Server-rendered, indexable list/item/article pages	100% core routes SSR
Accessibility	WCAG	2.1 AA for core flows
Cost	LLM + crawling cost per ranked product	Tracked; unit-economics positive
Observability	Tracing + LLM tracing + error tracking	End-to-end traces on all pipeline runs
Resilience	Pipeline is resumable; partial failure degrades gracefully	No full-site outage from pipeline failure

8. System Architecture

The architecture separates three planes: (a) a read-optimized web/edge plane that serves cached, precomputed content to users; (b) an AI research plane that continuously harvests, scores and ranks; and (c) a data plane shared by both. Users never wait on an agent; they read precomputed results, while Ask Flash performs retrieval over the precomputed corpus with streaming generation.

8.1 Logical components

- Edge/CDN + WAF (Cloudflare): caching, bot protection, DDoS, image delivery.
- Web app (Next.js, RSC): SSR/ISR pages, client interactions, BFF route handlers.
- Core API (NestJS or FastAPI): catalogue, lists, items, search, prices, auth, events.
- AI orchestration service: agent graphs, prompts, tool calls, scoring, fact-checking.
- Ingestion workers: crawlers, marketplace/feed connectors, YouTube/Reddit connectors, review extractors.
- Search service: lexical + vector index (OpenSearch + pgvector/Qdrant).
- Workflow engine (Temporal): durable, resumable long-running research runs.
- Data stores: Postgres (OLTP + pgvector), Redis (cache/queues), ClickHouse/BigQuery (analytics), S3/R2 (objects/media).
- Observability: OpenTelemetry, Sentry, Langfuse (LLM traces), Grafana, PostHog.
- Redirect service: affiliate deep-link generation, click attribution, interstitial disclosure.

8.2 Data flow (request path)

1. User requests a list/item page. Cloudflare serves a cached/ISR page; Next.js renders from the Core API or cache.
2. Ask Flash: request hits the AI service, which retrieves relevant chunks from the vector index, then streams an LLM answer with citations over SSE.
3. Outbound click: /go resolves the offer, logs attribution, shows disclosure, and 302-redirects to the merchant with affiliate params.

8.3 Data flow (research path)

1. Scheduler triggers a research run for a category/list via Temporal.
2. Scout agents harvest candidates from marketplaces, AI tools, web search and communities; entity resolution dedupes to canonical products.
3. Evidence agents collect reviews, YouTube transcripts, Reddit threads and expert lists; authenticity filters remove seeded reviews.
4. Extraction agents produce structured attributes, findings and downsides with source citations.
5. Scoring agents compute the Flash Score per list rubric; ranking agents order items; a fact-check agent validates consistency.
6. Price agents crawl stores for offers; results are written to the data plane and published to caches/ISR.
7. Every stage emits events to the Agent Hub and writes an auditable run record.

8.4 Key architectural decisions

Decision	Choice	Rationale
Rendering	Next.js App Router + RSC + ISR	SEO, fast TTFB, matches reference
Precompute vs on-demand	Precompute rankings/prices; retrieve on demand for chat	Users never wait on agents
Workflow	Temporal for long research runs	Durable, resumable, observable
Search	OpenSearch (lexical) + vector (pgvector/Qdrant) hybrid	Keyword precision + semantic recall
Storage	Postgres + S3 + Redis + ClickHouse	Proven, cost-effective at v1 scale
Media	S3/R2 origin + imgproxy/Cloudflare Images	On-the-fly transforms like i.flash.tech
Affiliate	Server-side /go redirect	Central attribution + params + disclosure

9. Technology Stack

Recommended stack, chosen to match the observed reference architecture while keeping a small team productive. Alternatives are listed where relevant.

9.1 Frontend

Concern	Recommendation	Notes / alternatives
Framework	Next.js 15 (App Router, RSC, TypeScript)	Matches flash.co; SSR/ISR for SEO
UI components	Tailwind CSS + Radix/shadcn, or MUI	Reference uses MUI + Tailwind + Emotion
State/data	TanStack Query + Zustand	Server state cache + light client state
Streaming	SSE (EventSource) for Ask Flash & Agent Hub	WebSocket if bidirectional needed
Fonts	next/font (Inter Tight, DM Sans, serif display)	Mirrors observed type system
PWA	Service worker, installable, offline shell	Mobile-first audience
Analytics	PostHog + GTM	Product analytics + tag management

9.2 Backend & API

Concern	Recommendation	Notes
Core API	NestJS (Node/TS) or FastAPI (Python)	TS shares types with frontend; Python eases ML
API style	REST + JSON, OpenAPI 3.1, SSE for streams	Simple, cacheable, easy to version
Auth	Clerk / Auth0 / Supabase Auth (or NextAuth)	OTP + OAuth, sessions, RBAC
Validation	Zod / Pydantic	Request/response contracts
Rate limiting	Token bucket at edge + API	Protect search/chat endpoints

9.3 AI, agents & data processing

Concern	Recommendation	Notes
LLMs	Frontier models (GPT-4o/o-series, Claude, Gemini)	Multi-model for cost/quality; router
Agent orchestration	LangGraph or custom graph + Temporal	Deterministic, resumable pipelines
Embeddings	text-embedding-3-large / open models	Store in pgvector/Qdrant
Reranking	Cross-encoder reranker (e.g. Cohere/BGE)	Improves RAG precision
RAG framework	LlamaIndex or LangChain (light usage)	Retrieval + citation plumbing
Crawling	Firecrawl / Playwright / Scrapy	JS-heavy pages, rate-limit aware
Connectors	YouTube Data/Transcript API, Reddit API, shopping feeds	Evidence sources

Authenticity	ML classifier + heuristics for seeded reviews	Spam/astroturf detection
LLM observability	Langfuse	Prompts, traces, cost, evals

9.4 Data & infrastructure

Concern	Recommendation	Notes
OLTP	PostgreSQL 16	Catalogue, users, rankings, offers
Vectors	pgvector (start) or Qdrant (scale)	Semantic retrieval
Cache/queue	Redis + BullMQ	Caching, rate limits, light jobs
Analytics/events	ClickHouse or BigQuery	Agent events, clicks, funnels
Object storage	AWS S3 or Cloudflare R2	Images, transcripts, artifacts
Search	OpenSearch / Elasticsearch	Lexical + faceting
Workflow	Temporal	Durable research runs
Containers	Docker + ECS/EKS or Vercel + Fly.io	Web on Vercel; workers on ECS
IaC	Terraform	Reproducible infra
Edge	Cloudflare (CDN, WAF, Images)	Matches observed edge

9.5 Observability, quality & tooling

Concern	Recommendation
Tracing/metrics	OpenTelemetry + Grafana/Prometheus
Errors	Sentry
LLM traces/evals	Langfuse + offline eval harness
Product analytics	PostHog
CI/CD	GitHub Actions; preview deploys; blue/green
Feature flags	Unleash or LaunchDarkly
Testing	Vitest/Jest, Playwright E2E, contract tests
Email/SMS	Resend/SES + MSG91/Twilio (OTP)

STACK SUMMARY (ONE LINE)

Next.js 15 + TypeScript + Tailwind on Cloudflare/Vercel; NestJS/FastAPI core API; Temporal + LangGraph agent pipelines; Postgres + pgvector + OpenSearch + Redis + ClickHouse + S3/R2; multi-LLM with Langfuse observability; affiliate redirect service for monetization.

10. Data Model

Core entities and key fields. IDs are opaque strings; all timestamps UTC. The model supports many-to-many between products and lists, and between products and offers.

Entity	Key fields	Relationships
Vertical	id, name, slug, order	has many Categories
Category	id, vertical_id, parent_id, name, slug, path	tree; has many Lists
Brand	id, name, slug, badges, logo	has many Products
Product	id, brand_id, title, description, specs(jsonb), images[], aliases[]	many-to-many Lists; has many Offers
Merchant	id, name, slug, logo, affiliate_config	has many Offers
Offer	id, product_id, merchant_id, price, mrp, currency, coupon, in_stock, deep_link, checked_at	belongs to Product/Merchant
List	id, category_id, title, intro, rubric_version, ranked_count, last_ranked_at	has many ListItems
ListItem	list_id, product_id, rank, score, delta, verdict, best_for[]	join List/Product
Evidence	id, product_id, source_type, source_url, title, excerpt, published_at, authenticity_score	belongs to Product
Finding	id, product_id, list_id, type(pro con), text, evidence_ids[]	belongs to Product
Score	id, product_id, list_id, rubric_version, total, breakdown(jsonb), computed_at	belongs to Product/List
ResearchRun	id, run_no, scope, status, started_at, finished_at, stats(jsonb)	has many AgentEvents
AgentEvent	id, run_id, agent_type, message, item_id, created_at	belongs to ResearchRun
User	id, auth_provider, email/phone_hash, created_at	has many Sessions/Saves
Click	id, user_id?, item_id, offer_id, list_id?, session_id, ts, merchant	attribution
Article	id, vertical, section, slug, title, body, citations[], product_ids[]	links to Lists/Products

10.1 Ranking-integrity invariants (data-level)

- No field in Score or ListItem may be written by a monetization path; write access is restricted to the research pipeline service identity.
- Every Score row references the rubric_version used, enabling historical reproduction.
- Every Finding references at least one Evidence row; orphan findings fail validation.
- Click attribution is append-only and never mutates ranking data.

11. API Specification

REST over HTTPS, JSON bodies, versioned under /v1. Auth via Bearer JWT (or session cookie for web). Streaming endpoints use Server-Sent Events. All list endpoints are cursor-paginated. Errors follow RFC 9457 (application/problem+json).

11.1 Conventions

Aspect	Convention
Base URL	https://api.example.com/v1
Auth	Authorization: Bearer <token>; guest tokens allowed for read + limited chat
Pagination	?limit=25&cursor=<opaque>; response has next_cursor
Filtering	?price_max=1000&merchant=amazon&attr=no_white_cast
Sorting	?sort=rank score price_asc price_desc recent
Idempotency	Idempotency-Key header on POST
Rate limits	429 with Retry-After; per-IP and per-token buckets
Caching	ETag + Cache-Control on public GETs; ISR at web layer
Versioning	URL version; additive changes only within a major

11.2 Public endpoints

Method & path	Purpose	Key params
GET /categories	Taxonomy tree	vertical, depth
GET /categories/{slug}	Category detail + lists	—
GET /discovery/{id}	Attribute cluster landing	—
GET /lists	List collection	category, vertical, sort, cursor
GET /lists/{id}	Ranked list with provenance	—
GET /lists/{id}/items	Paginated ranked items	limit, cursor, filters
GET /items/{id}	Product detail (score, verdict, specs)	—
GET /items/{id}/research	Findings, downsides, sources	—
GET /items/{id}/prices	Multi-store offers	—
GET /items/{id}/rankings	Lists this item ranks in	—
GET /brands/{slug}	Brand profile	—
GET /search	Keyword + hybrid search	q, type, filters, cursor
POST /ask	Ask Flash RAG (SSE stream)	body: {query, context?}

POST /analyze	Analyze pasted URL/reel	body: {url}
GET /articles	Editorial index	vertical, section, cursor
GET /articles/{slug}	Editorial article	—
GET /agents/events	Live agent feed (SSE)	type, window, since
GET /agents/runs/{runId}	Run detail	—
POST /events	Client analytics ingestion	body: event batch
GET /go	Affiliate redirect	u, item, offer, list, sig
GET /me	Current user profile	auth
GET /me/saves	Saved items/lists	auth

11.3 Sample: GET /lists/{id}

```
GET /v1/lists/1677 HTTP/1.1
Host: api.example.com

200 OK
{
  "id": "1677",
  "title": "Sunscreens",
  "category_path": ["Beauty", "Skincare", "Sunscreens"],
  "intro": "The best-scored sunscreens, verified from real evidence.",
  "ranked_count": 833,
  "last_ranked_at": "2026-08-24T00:00:00Z",
  "provenance": {
    "expert_analysis": 6155, "youtube": 4436,
    "reviews": 11019, "reddit_threads": 1023
  },
  "items": [
    {
      "rank": 1, "product_id": "p_9f2c",
      "title": "La Roche-Posay Anthelios Oil Control SPF50+",
      "brand": "La Roche-Posay", "score": 89,
      "best_price": { "amount": 1400, "currency": "INR", "merchant": "amazon" },
      "rank_delta": 2
    }
  ],
  "next_cursor": "eyJvIjoyNX0"
}
```

11.4 Sample: GET /items/{id}/prices

```
GET /v1/items/2760/prices HTTP/1.1

200 OK
{
  "item_id": "2760",
  "lowest": { "amount": 600, "currency": "INR", "merchant": "myntra" },
  "offers": [
    {
      "merchant": "myntra", "amount": 600, "mrp": 800,
      "coupon": "Extra 25% off with selected bank cards",
      "in_stock": true, "checked_at": "2026-10-05T06:00:00Z",
      "deep_link": "/go?u=...&offer=o_1&sig=..."
    },
    {
      "merchant": "amazon", "amount": 629, "mrp": 699,
      "in_stock": true, "checked_at": "2026-10-05T06:00:00Z",
      "deep_link": "/go?u=...&offer=o_2&sig=..."
    }
  ]
}
```

11.5 Sample: POST /ask (SSE)

```
POST /v1/ask HTTP/1.1
Content-Type: application/json
Accept: text/event-stream

{ "query": "best sunscreen for oily acne-prone skin under 700" }

event: token
data: {"text":"For oily, acne-prone skin under"}

event: token
data: {"text":" Rs.700, three picks stand out."}

event: citation
data: {"item_id":"2760","list_id":"1680","source":"research"}

event: done
data: {"answer_id":"a_91","tokens":184,"cost_usd":0.004}
```

11.6 Sample: affiliate redirect

```
GET /go?u=https%3A%2F%2Fwww.amazon.in%2Fdp%2FB0XXXX
&item=2760&offer=o_2&list=1677&sig=abc123 HTTP/1.1

302 Found
Location: https://www.amazon.in/dp/B0XXXX?tag=flashai-21&linkCode=ll2
Set-Cookie: attr=<opaque>; Path=/; HttpOnly; Secure; SameSite=Lax
```

The redirect service: (1) validates the signed u/item/offer tuple, (2) resolves the merchant affiliate template, (3) appends affiliate params, (4) writes an immutable Click row, (5) renders a disclosure interstitial when required, then 302s.

11.7 Internal / pipeline endpoints

Method & path	Purpose	Auth
POST /internal/runs	Start a research run	Service mTLS
POST /internal/rankings/rerank	Trigger re-ranking for a list	Service mTLS
POST /internal/scores	Write scores (pipeline identity only)	Service mTLS
POST /internal/evidence	Upsert evidence/findings	Service mTLS
POST /internal/prices	Bulk upsert offers	Service mTLS
GET /internal/runs/{id}/audit	Run audit trail	Admin

API DESIGN RULES

Public read APIs are cache-friendly and can be served from the edge. All writes to ranking/scoring data are restricted to internal service identities over mTLS. Monetization code may only read ranking data and write click/attribution data, enforcing the zero-paid-ranking invariant at the authorization layer.

12. AI Agent Research & Ranking Pipeline

The research pipeline is the product's core differentiator. It is a durable, multi-stage workflow executed by specialized agents, each with a narrow job, structured outputs and full traceability. Runs are numbered and resumable.

12.1 Stage overview

#	Stage	Agent	Input	Output
1	Taxonomy & query planning	Planner	Category/list intent	Sub-queries, seed brands, attribute schema
2	Candidate harvesting	Scout	Sub-queries	Candidate products from many sources
3	Entity resolution	Resolver	Raw candidates	Canonical deduped products
4	Evidence collection	Researcher	Products	Reviews, videos, threads, expert lists
5	Authenticity filtering	Verifier	Raw reviews	Kept/discarded + authenticity score
6	Attribute extraction	Extractor	Evidence text	Structured attributes, findings, downsides
7	Scoring	Scorer	Attributes + rubric	0-100 Flash Score + breakdown
8	Ranking & re-ranking	Ranker	Scores + list context	Ordered ListItems + deltas
9	Fact-check / critic	Critic	Rankings + evidence	Flags, corrections, approval
10	Price crawl	Pricer	Products + merchants	Live offers per store
11	Publish	Publisher	Approved results	Cached pages, ISR, events
12	Continuous re-run	Scheduler	Freshness policy	New runs on cadence/triggers

12.2 Agent design principles

- **Narrow scope:** one agent, one job, one structured output schema (JSON validated).
- **Evidence-first:** every extracted claim carries source URLs and excerpts; unsourced claims are rejected.
- **Idempotent & resumable:** each stage writes checkpoints; failures resume without redoing expensive work.
- **Deterministic where it matters:** fixed rubrics, versioned prompts, seed control, and caching to make scores reproducible.
- **Multi-model routing:** cheap models for extraction, strong models for reasoning/critique.
- **Human-in-the-loop:** critic flags route to review queues; publish can require approval for sensitive categories.

- Cost guardrails: token/request budgets per run; incremental re-runs only for changed evidence.

12.3 Scoring rubric (illustrative)

Dimension	Weight	Signals
Efficacy / performance	30%	Independent tests, expert consensus, measured outcomes
Evidence strength	20%	Number & quality of independent sources; authenticity
User satisfaction	15%	Verified buyer sentiment after de-duplication
Value for money	15%	Price vs performance vs category median
Composition / build quality	10%	Ingredients/materials, specs, certifications
Availability & support	10%	Stock, warranty, service reputation

The rubric is list-aware: a 'budget' list reweights value-for-money upward; a 'sensitive skin' list reweights composition and evidence upward. Rubric version is stored with every score.

12.4 Freshness policy

Content	Cadence	Trigger
Prices (top items)	Every 6 hours	Price change > 5%
Rankings (hot lists)	Weekly	New evidence / new launch
Rankings (long tail)	Every 14-30 days	Scheduled
Evidence corpus	Continuous	New review/video/thread
Fact-check	Every run	Score/rank inconsistency

13. Complete Project Flow - Zero to Deployment

This section is the master execution guide: the exact order in which to build the project, what to produce at each step, and how the pieces connect. It is written so that a student or small team can follow it top to bottom. Each phase produces a concrete artifact that the next phase consumes; nothing is a stub.

13.1 The master flow at a glance

PHASE 0	Setup	->	toolchain, repo, local infra, env keys
PHASE 1	Data	->	taxonomy, connectors, canonical catalogue
PHASE 2	AI Pipeline	->	harvest -> evidence -> score -> rank -> price
PHASE 3	Backend API	->	REST endpoints, auth, search, redirect
PHASE 4	Frontend	->	browse, list, item, search, Ask, agent hub
PHASE 5	Test	->	unit, contract, E2E, load, AI evals
PHASE 6	Deploy	->	CI/CD, edge, monitoring, cost controls
PHASE 7	Operate	->	re-runs, freshness, growth, integrity audit

GOLDEN RULE

Build in vertical slices: data -> API -> UI -> instrumentation. Never build a large horizontal layer (e.g. all scrapers) before any page renders. A thin end-to-end path that works beats a perfect layer that is not wired up.

13.2 Phase 0 - Environment and repository setup

Goal: every developer can run the full stack locally in under 30 minutes. Deliverable: a documented, reproducible dev environment.

1. Install the toolchain: Node.js 20+, pnpm, Python 3.11+, Docker Desktop, Git. Optional: Postgres/Redis/OpenSearch via Docker (no native installs needed).
2. Create the monorepo (pnpm workspaces or Turborepo) with the structure shown below.
3. Add docker-compose for local Postgres (with pgvector), Redis and OpenSearch.
4. Create .env.example with every key named and documented; never commit real keys.
5. Add lint/format (ESLint + Prettier + Ruff), commit hooks (Husky), and a root README.

```
shopclone/  
  apps/  
    web/      # Next.js 15 frontend (App Router, RSC)  
    api/      # NestJS (TS) or FastAPI (Python) core API  
    workers/  # ingestion + agent workers (Temporal activities)  
  packages/  
    ui/       # shared React components + design tokens  
    types/    # shared TS types + Zod schemas (single source of truth)  
    prompts/  # versioned prompt templates (reviewed like code)  
  infra/     # docker-compose, terraform, k8s manifests  
  docs/      # PRD, ADRs, OpenAPI spec, runbooks
```

```
# bootstrap the local environment
git clone <your-repo> && cd shopclone
pnpm install
cp .env.example .env          # fill keys (see Section 16)
docker compose up -d postgres redis opensearch
pnpm db:migrate && pnpm db:seed
pnpm dev                      # web :3000 | api :4000 | workers
```

13.3 Phase 1 - Data foundation

Goal: a canonical catalogue with a taxonomy and at least one working connector. Deliverable: browse-ready categories and item records in Postgres.

1. Model the taxonomy (Vertical > Department > Category > Attribute) and seed 3 verticals with 10-20 categories each.
2. Implement the database schema (Section 10) and migrations for products, merchants, offers, lists, evidence, findings, scores, runs and events.
3. Build the first connector: an affiliate product feed or a compliant scraper for one merchant. Normalize into Product + Offer.
4. Implement entity resolution (dedupe the same product across merchants) using normalized titles + brand + identifiers, with a Union-Find or blocking + similarity approach.
5. Build the image pipeline: fetch, transform (resize/webp) and store to S3/R2 with a CDN URL.
6. Seed enough items to render a real category page end-to-end.

MILESTONE M2

A category page renders real products with real prices from the database, served by the API.

13.4 Phase 2 - AI research and ranking pipeline

Goal: agents produce an evidence-backed ranked list. Deliverable: 3 fully researched, published lists.

1. Implement the orchestration skeleton with Temporal: a workflow with resumable, idempotent activities.
2. Build Scout: generate candidate sub-queries and harvest candidates from marketplace feeds, search APIs and communities.
3. Build Resolver: dedupe candidates to canonical products (this reuses Phase 1 entity resolution).
4. Build Researcher: collect evidence (reviews, YouTube transcripts, Reddit threads, expert lists) with source URLs and excerpts.
5. Build Verifier: classify seeded/fake reviews and store an authenticity score; discard low-trust evidence.
6. Build Extractor: produce structured attributes, findings (pros) and downsides, each linked to evidence IDs (JSON schema validated).
7. Build Scorer: apply the versioned rubric to compute a 0-100 Flash Score with a stored breakdown.
8. Build Ranker: order items for a list, compute rank deltas vs the previous run.
9. Build Critic: fact-check consistency; flag items lacking attribute scoring or unsupported claims.
10. Build Pricer: crawl/refresh offers across merchants; store price, MRP, coupon, stock and checked_at.
11. Build Publisher: write results to Postgres, update search/vector indexes, invalidate caches/ISR, emit AgentEvents.

12. Create an evaluation set (golden questions and expected picks) and block releases on regression.

MILESTONE M3

Three categories are researched, scored, ranked and published entirely by the pipeline, with citations.

13.5 Phase 3 - Backend API

Goal: the API that powers all client surfaces. Deliverable: an OpenAPI-documented, tested API.

1. Scaffold the API (NestJS/FastAPI) with health checks, structured logging and OpenAPI generation.
2. Implement read endpoints: categories, lists, list items, items, research, prices, rankings, brands.
3. Implement search: lexical (OpenSearch) first, then add hybrid vector retrieval (pgvector/Qdrant) and reranking.
4. Implement Ask Flash: retrieve top chunks -> build prompt -> stream tokens and citations over SSE.
5. Implement auth (OTP/OAuth), sessions and per-user saves/history.
6. Implement the /go affiliate redirect: signature validation, attribution write, disclosure, 302.
7. Add caching (ETag/Cache-Control), cursor pagination, rate limiting and error handling (RFC 9457).

MILESTONE M4

Search and Ask Flash return grounded, cited answers; the API is fully documented.

13.6 Phase 4 - Frontend

Goal: the user-facing product. Deliverable: browse, list, item, search and hub pages live.

1. Build the design system (tokens, type scale, layout, components) and the shell (header, nav, bottom bar).
2. Build the browse page and category navigation from the taxonomy API.
3. Build the list page (ranked items, provenance counts, filters, re-rank date).
4. Build the item page (score, verdict, findings/downsides, specs, rankings, prices) and the compare-prices page.
5. Build search with autocomplete (Trie/prefix) and result grouping (products, lists, articles).
6. Build Ask Flash chat with streaming, citations and product cards; build the live Agent Hub feed (SSE).
7. Add SEO (metadata, canonical, structured data), performance budgets and accessibility.

MILESTONE M5

A user can browse -> read a list -> open an item -> compare prices -> click through to buy, and ask a question.

13.7 Phase 5 - Integration, testing and evaluation

Goal: confidence to launch. Deliverable: green test suite and documented eval results.

Test type	What it covers	Tooling
Unit	Scoring math, ranking comparators, parsers, entity resolution	Vitest / Jest / pytest

Contract	API request/response shapes against OpenAPI	Schemathesis / Pact
Integration	DB + queue + cache wiring, pipeline activities	Testcontainers
E2E	Critical user journeys in a real browser	Playwright
Load	Search/chat/list under peak concurrency	k6 / Locust
AI evals	Extraction accuracy, score stability, RAG citation rate	Custom harness + Langfuse
Security	Auth, redirect signing, injection, rate limits	OWASP ZAP / manual review

13.8 Phase 6 - Deployment, monitoring and operations

Goal: reliable production. Deliverable: automated deploys with observability and cost controls.

1. Containerize services; set up CI/CD (GitHub Actions) with preview environments and blue/green or canary deploys.
2. Provision managed Postgres, Redis, OpenSearch and object storage; put Cloudflare in front (CDN, WAF, image delivery).
3. Run web on Vercel or containers; run workers on a separate autoscaling pool so pipeline bursts never affect the site.
4. Wire OpenTelemetry traces, Sentry errors, Langfuse LLM traces, PostHog analytics and Grafana dashboards/alerts.
5. Add cost controls: model routing, prompt/response caching, incremental re-runs, per-run token budgets.
6. Write runbooks for pipeline failure, price drift, and ranking-integrity incidents; schedule backups.

MILESTONE M7

v1 GA with SLOs met (99.9% uptime, LCP <= 2.5s) and positive unit economics.

13.9 Daily development workflow

1. Pull latest main; create a feature branch named feat/<area>-<short-desc>.
2. Write the API contract (OpenAPI/Zod) first, then implement API, then UI, then tests.
3. Run lint, typecheck and tests locally; open a pull request with a description and screenshots.
4. CI runs tests + preview deploy; a reviewer approves; merge to main; CD deploys to staging then production.
5. Tag releases (semver); keep a changelog; monitor dashboards for 30 minutes post-deploy.

13.10 End-to-end flow diagrams

READ REQUEST FLOW

User -> Cloudflare (cache/WAF) -> Next.js RSC/ISR
-> Core API (cache hit?) -> Postgres/OpenSearch -> Response

ASK FLASH FLOW

User -> /ask -> retrieve top-k chunks (vector+lexical) -> rerank
-> LLM prompt -> stream tokens + citations (SSE) -> UI

OUTBOUND / MONETIZATION FLOW

Click -> /go?u=...&item=..&sig=.. -> verify signature
-> write Click row -> disclosure -> 302 to merchant (affiliate params)

RESEARCH RUN FLOW (durable, resumable)

Scheduler -> Temporal workflow
-> Scout -> Resolver -> Researcher -> Verifier
-> Extractor -> Scorer -> Ranker -> Critic -> Pricer -> Publisher
-> Postgres + OpenSearch + pgvector
-> AgentEvents (SSE) + cache/ISR invalidation

13.11 Deliverables checklist

Artifact	Owner	Done when
Source code monorepo	All engineers	Builds and tests pass in CI
OpenAPI spec	Backend	All public endpoints documented
DB schema + migrations	Backend/Data	Reproducible from scratch
Agent pipeline + prompts	AI/ML	3 categories researched end-to-end
Eval report	AI/ML	Extraction/score/RAG metrics recorded
Frontend app	Frontend	All core journeys pass E2E
Infra as code	DevOps	One-command environment provisioning
Runbooks + dashboards	DevOps	Alerts firing to on-call
This PRD + ADRs	PM/Tech Lead	Approved and versioned

14. Build Roadmap, Milestones & Delivery Process

The delivery plan is organized into phases with clear exit criteria. Assumes a small, senior cross-functional team. Durations are indicative. Section 13 is the step-by-step how; this section is the schedule.

Phase	Duration	Scope	Exit criteria
P0 Discovery	2 weeks	Taxonomy, rubric, source inventory, legal review of crawling/affiliate	Signed-off PRD + data-source plan
P1 Foundation	4 weeks	Infra, auth, catalogue + entity resolution, item/list pages, redirect service	Item/list pages live with real offers
P2 AI Pipeline MVP	6 weeks	Harvest -> evidence -> extract -> score -> rank -> critic for 3 categories	First 10 evidence-backed lists published
P3 Search + Ask Flash	4 weeks	Hybrid search, RAG chat with citations, paste-link analysis	Search & chat in beta with >=90% cited answers
P4 Prices + Monetization	3 weeks	Multi-store price crawl, offers UI, affiliate deep links, attribution	97% price accuracy; revenue tracking live
P5 Live Hub + Editorial	3 weeks	Agent Hub SSE feed, run logs, The Cut content pipeline	Live hub streaming; 20 articles published
P6 Hardening & Scale	ongoing	Perf, SEO, accessibility, cost tuning, autoscaling, eval harness	SLOs met; unit economics positive

14.1 Milestones (M)

- M1 (end P0): Requirements, rubric v1, source/legal plan approved.
- M2 (end P1): Catalogue + item/list pages + affiliate redirect in staging.
- M3 (end P2): 3 categories fully researched and published by agents.
- M4 (end P3): Search + Ask Flash in beta.
- M5 (end P4): Price comparison + affiliate revenue instrumentation.
- M6 (end P5): Public Agent Hub + editorial launch.
- M7 (end P6): v1 GA; SLOs and unit economics validated.

14.2 Delivery workflow

1. Discovery: define taxonomy, rubric and source list; run legal/compliance review of data acquisition and affiliate terms.
2. Build vertical slices: for each feature, ship data -> API -> UI -> instrumentation together so nothing is a stub.
3. Evaluate continuously: maintain an offline eval set for extraction, scoring and RAG; block releases on regression.
4. Shadow-run the pipeline: run agents on categories before launch and compare scores to expert judgement.
5. Launch behind flags; ramp categories; monitor cost per ranked product and click-through.

14.3 Definition of done (per feature)

- API contract (OpenAPI) merged and documented.
- Unit + contract + E2E tests passing in CI.
- Instrumentation (analytics + traces) emitting.
- Accessibility and performance budgets met.
- Rollback/feature-flag path defined.
- Runbook + alert thresholds added.

15. Knowledge, Concepts & DSA Required

This section lists everything a builder (or a student submitting this as a project) needs to know, organized from foundations to advanced. The DSA subsection maps specific data structures and algorithms to concrete parts of this product, which is what makes the project defensible as a Data Structures & Algorithms / systems project.

15.1 Prerequisite programming knowledge

- One backend language well: JavaScript/TypeScript (Node) or Python. You will read and write both eventually.
- SQL and relational modeling: SELECT/JOIN/GROUP BY, indexes, transactions, migrations.
- Web fundamentals: HTTP methods, status codes, headers, cookies, REST, JSON.
- HTML/CSS and a frontend framework (React) with basic state management.
- Git and the command line; reading documentation and error messages.
- Basic async/concurrency: promises/async-await or asyncio; queues and background jobs.

15.2 DSA concepts mapped to this project

Each structure/algorithm below has a real, non-contrived place in the product. Use this table directly in a DSA project report to justify every algorithmic choice.

Data structure / algorithm	Where it is used in this project	Why it fits
Array / dynamic array	Ranked result sets, batches of candidates, page data	Cache-friendly, indexable, $O(1)$ access
Hash map / dictionary	Product lookup by id, cache, config, token counts	$O(1)$ average lookup/insert
Hash set	Crawler seen-set, source de-duplication, tag membership	$O(1)$ membership; removes duplicates
Sorting + comparators	Ordering products by score, price, rank; multi-key sort	Stable, predictable ordering; $O(n \log n)$
Heap / priority queue	Top-K picks per category, trending, merging K sorted merchant feeds	$O(n \log k)$ Top-K; $O(\log n)$ push/pop
Binary search / BST	Price-range filters, sorted index lookups, range queries	$O(\log n)$ search on sorted data
Trie / prefix tree	Search autocomplete, brand/category suggestions	$O(m)$ prefix lookup (m = key length)
Inverted index	Keyword search over products, lists and articles	Fast term $\rightarrow$ document retrieval
Graph (DAG)	Category taxonomy, agent pipeline dependency graph, product relations	Models hierarchy and dependencies

BFS / DFS	Category traversal, related-product expansion, crawl frontier	$O(V + E)$ traversal
Topological sort	Ordering pipeline stages; resolving dependencies	Correct execution order for a DAG
Union-Find (disjoint set)	Merging duplicate products across merchants (entity resolution)	Near $O(1)$ union/find with path compression
Dynamic programming	Edit distance / fuzzy matching, diffing rank changes, optimal batching	Solves overlapping-subproblem optimization
Greedy	Crawl scheduling, source selection, budget allocation	Fast near-optimal decisions
Sliding window	Rate limiting, rolling event counts for the Agent Hub histogram	$O(n)$ over a stream
LRU cache	Caching hot pages, queries and embeddings	$O(1)$ get/put with eviction
Bloom filter	Space-efficient crawler URL de-duplication	$O(k)$ membership with tiny memory
Consistent hashing	Sharding cache / vector index across nodes	Minimal rebalancing on scale
MinHash / LSH	Near-duplicate review and product detection	Approximate similarity at scale
TF-IDF / BM25	Lexical relevance for search	Ranked keyword retrieval
Cosine similarity + ANN	Semantic search, vector dedup (HNSW/IVF index)	Fast approximate nearest neighbours
Weighted scoring / rank fusion	Flash Score; combining expert, review and price signals	Transparent, tunable ranking
PageRank-style centrality	Source and brand authority weighting	Graph-based trust signal
Elo / Bradley-Terry	Pairwise product comparisons into a rating	Principled comparison model
MapReduce / streaming aggregation	Analytics over click and agent events	Scalable batch/stream processing
Fenwick / segment tree (advanced)	Leaderboards, price-range aggregate queries	$O(\log n)$ range queries/updates

```
# Top-K products per category using a min-heap          ->  $O(n \log k)$ 
heap = MinHeap(key=score)
for p in products:
    heap.push(p)                #  $O(\log k)$ 
    if heap.size > K: heap.pop() # evict worst
return heap.items_sorted_desc()

# Entity resolution with Union-Find                      -> near  $O(1)$  per op
for a, b in candidate_pairs:
    if similar(a, b): union(a, b)
clusters = group_by(find(p) for p in products)
```

15.3 Complexity quick reference (use in the report)

Operation	Structure / algorithm	Average	Worst
-----------	-----------------------	---------	-------

Key lookup	Hash map	$O(1)$	$O(n)$
Top-K selection	Min-heap of size k	$O(n \log k)$	$O(n \log k)$
Full sort	Comparison sort	$O(n \log n)$	$O(n \log n)$
Prefix search	Trie	$O(m)$	$O(m)$
Union / find	Union-Find + path compression	$O(\alpha(n))$	$O(\alpha(n))$
Graph traversal	BFS / DFS	$O(V + E)$	$O(V + E)$
Dependency order	Topological sort	$O(V + E)$	$O(V + E)$
Lexical query	BM25 over inverted index	$O(\text{terms} \times \text{postings})$	$O(\text{terms} \times \text{postings})$
Semantic search	ANN (HNSW)	$\sim O(\log n)$	$\sim O(\log n)$ approx
Edit distance	Dynamic programming	$O(n \times m)$	$O(n \times m)$
Rate limit check	Sliding window	$O(1)$ amortized	$O(1)$ amortized
Cache access	LRU (hash map + list)	$O(1)$	$O(1)$
Membership	Bloom filter	$O(k)$	$O(k)$
Range filter	Binary search on sorted data	$O(\log n)$	$O(\log n)$

15.4 System design concepts

- Client-server and stateless APIs; REST resource design and versioning.
- Caching layers: browser, CDN/edge, application, database; cache invalidation.
- Load balancing, horizontal vs vertical scaling, autoscaling worker pools.
- Database: indexing, normalization vs denormalization, transactions, replication, sharding.
- CAP theorem and eventual consistency; choosing consistency where trust requires it (rankings, prices).
- Message queues and pub/sub; asynchronous jobs; idempotency and exactly-once effects.
- Rate limiting, backpressure, circuit breakers, graceful degradation.
- Observability: logs, metrics, traces; SLIs/SLOs and alerting.
- CI/CD, containers, infrastructure as code, blue/green and canary releases.
- Security: authentication vs authorization, least privilege, signing, secrets management.

15.5 AI / ML concepts

- LLMs: tokens, context windows, temperature, cost per token, model routing.
- Embeddings and vector databases; chunking strategies; metadata filtering.
- RAG: retrieval -> rerank -> grounded generation with citations.
- Prompt engineering: system prompts, few-shot examples, structured output / JSON mode.
- Tool/function calling; agents; ReAct loops; planning; memory; multi-agent orchestration.
- Evaluation: golden datasets, precision/recall, score stability, citation rate, regression gates.
- Hallucination, grounding, guardrails, and human-in-the-loop review.
- Classification for spam / fake-review detection; authenticity scoring.
- When to prompt vs fine-tune vs distill; using smaller/cheaper models where possible.

15.6 Frontend / web concepts

- Rendering models: CSR, SSR, SSG, ISR, and React Server Components; hydration.
- Component design, props/state, hooks, routing, data fetching and caching.
- State management (server state vs client state); optimistic UI.
- Real-time transport: SSE vs WebSockets.
- Accessibility (WCAG), responsive/mobile-first design, SEO and structured data.
- Web performance metrics: LCP, CLS, INP; code splitting and image optimization.
- Progressive Web App basics: installability, offline shell.

15.7 Data engineering concepts

- ETL vs ELT; batch vs streaming; incremental and change-data-capture loads.
- Data modeling: OLTP normalization vs analytical star schema.
- Orchestration with DAGs; retries, idempotency, backfills.
- Data quality: validation, de-duplication, freshness, lineage.
- Warehousing/analytics (ClickHouse/BigQuery) and event schemas.
- Partitioning and indexing for query performance.

15.8 Suggested learning path (8 weeks, part-time)

Week	Focus	Outcome
1	HTTP, REST, SQL, Git, project setup	Local stack running; first API endpoint
2	DSA refresher: arrays, hashing, sorting, heaps, graphs	Can choose the right structure for a feature
3	Database schema, migrations, indexing, connectors	Real products and offers in the DB
4	Entity resolution, de-duplication, image pipeline	Canonical catalogue rendered on a page
5	LLMs, embeddings, RAG, prompt/structured output	Working Ask Flash prototype
6	Agents, orchestration, scoring and ranking	First evidence-backed ranked list
7	Frontend pages, search, chat, agent hub	End-to-end user journey works
8	Testing, deployment, monitoring, cost controls	Deployed MVP with SLOs and dashboards

15.9 If this is a DSA / academic project

- Frame the project around algorithm choice: show the naive approach, its complexity, then the chosen data structure/algorithm and its improvement.
- Include a complexity analysis section (use the table in 15.3) and justify Top-K heap vs full sort, Union-Find vs naive merge, ANN vs brute-force vector search.
- Document the pipeline as a DAG and map it to topological ordering; document the taxonomy as a graph and map traversal to BFS/DFS.
- Provide a DFD (Level 0/1), ER diagram, use-case diagram, sequence diagrams and the architecture diagram.
- Include a results section with measured latency/throughput and, for AI, extraction accuracy and citation rate.

- Be ready for viva questions on why each structure was chosen, space/time trade-offs, and how the system scales.

16. API & Services Catalog (Free vs Paid)

Every external service this project can use, with an indicative free tier and paid model. Prices and limits change frequently: always confirm on the provider's pricing page. 'Free tier' means a usable no-cost allowance for development; production traffic will almost always require a paid plan or self-hosting. Affiliate programs are revenue sources, not costs.

COST RULE OF THUMB

For a student/zero-budget build, combine free tiers with self-hosted open-source (Ollama or a free LLM tier, pgvector, Postgres full-text, Playwright/Scrapy, local Docker) and deploy the web app on a free host. Move to paid managed services only when traffic or reliability demands it.

16.1 LLM, embeddings & AI services

Service	Purpose	Free tier	Paid (indicative)	Alternatives
OpenAI API	Reasoning, generation, embeddings	Trial credits only; no permanent free tier	Per 1M tokens (input/output)	Anthropic, Gemini, Mistral, Groq, OpenRouter
Google Gemini API	Generation, embeddings, vision	Yes, rate-limited free tier	Pay-as-you-go / Vertex AI	OpenAI, Claude, Llama
Anthropic Claude API	Long-context reasoning, critique	Console trial credits	Per 1M tokens	OpenAI, Gemini
Groq	Very fast open-model inference	Yes, generous free tier	Pay-per-token	Together AI, Fireworks, Replicate
OpenRouter	Unified gateway to many models	Some free models	Pass-through token pricing	Direct provider SDKs
Ollama (self-host)	Run open LLMs locally	Fully free (your hardware)	Electricity/hardware only	llama.cpp, vLLM, LM Studio
Cohere	Embeddings + rerank	Trial keys	Per call / seat	Voyage, Jina, BGE (OSS)
Voyage AI	High-quality embeddings	Limited free tier	Per token	OpenAI, Cohere, BGE
Hugging Face Inference	OSS models, embeddings	Free tier	Inference Endpoints (hourly)	Replicate, Together

16.2 Vector databases & search

Service	Purpose	Free tier	Paid (indicative)
pgvector (extension)	Vector search inside Postgres	Free OSS	Only DB hosting cost
Qdrant	Dedicated vector DB	Free OSS + free cloud tier	Cloud usage tiers
Weaviate	Vector DB + hybrid search	Free OSS + sandbox	Cloud usage tiers

Milvus / Zilliz	Large-scale vector search	Free OSS; Zilliz free tier	Cloud usage tiers
Chroma	Lightweight local vector store	Free OSS	Cloud (beta)
Pinecone	Managed vector DB	Free starter index	Usage/serverless tiers
OpenSearch / Elasticsearch	Lexical + faceted search	Free OSS (self-host)	Elastic Cloud / AWS managed
Meilisearch / Typesense	Typo-tolerant search	Free OSS	Cloud from low monthly fee
Algolia	Hosted search-as-a-service	Free tier (record/ops limited)	Usage-based, premium
Postgres full-text search	Basic lexical search	Free (included)	No extra cost

16.3 Crawling, web & data-source APIs

Service	Purpose	Free tier	Paid (indicative)
Firecrawl	Crawl/convert pages to clean text	Yes, limited credits	Credit packs / monthly
Playwright / Puppeteer / Scrapy	Self-hosted scraping	Free OSS	Your compute cost only
ScraperAPI / ScrapingBee	Proxy + anti-bot scraping	Trial credits	Per request / plan
Bright Data / Zyte	Enterprise proxy + scraping	Trial	Usage-based, premium
Apify	Cloud actors / crawlers	Monthly free credits	Usage-based
Google Custom Search JSON API	Web search results	100 queries/day	Per 1,000 queries
Brave Search API	Independent web search	Free tier (rate-limited)	Per 1,000 queries
SerpAPI	Search engine results	100 searches trial	Per search / plan
Tavily / Exa	LLM-oriented web search	Free tier	Usage-based
YouTube Data API	Video metadata + search	10,000 units/day free	Higher quota via request
youtube-transcript-api (OSS)	Fetch video transcripts	Free OSS	None (self-host)
Reddit API	Threads, comments, sentiment	Free with rate limits	Enterprise tiers
Google Content API for Shopping	Product catalogue data	Free	None

16.4 Commerce & affiliate programs (India) - these earn revenue

Program	Purpose	Cost to join	How you earn
Amazon Associates / PA-API	Amazon product data + deep links	Free (approval required)	Commission per qualifying sale
Flipkart Affiliate	Flipkart product links	Free (approval required)	Commission per sale

Cuelinks / EarnKaro	Aggregator across many merchants	Free	Commission share
vCommission / Admitad	Affiliate network (many brands)	Free	Commission share
Impact / CJ / ShareASale	Global affiliate networks	Free to join	Commission per conversion
Direct brand programs	Higher rates, direct terms	Free (negotiated)	Commission / CPA

Note: affiliate links must be disclosed (see Section 20). Ranking integrity is a hard invariant: affiliate/commission data must never influence scores or order.

16.5 Platform, data, auth, hosting & operations

Category	Service options	Free tier	Paid (indicative)
Auth	Clerk, Auth0, Supabase Auth, Firebase Auth, NextAuth (OSS)	Most have free tiers	Per MAU / seat
Managed Postgres	Supabase, Neon, Railway, Render, PlanetScale (MySQL)	Free tiers with limits	Usage-based
Cache / queue	Upstash Redis, Redis Cloud, BullMQ (OSS)	Free tiers; OSS free	Usage-based
Workflow / orchestration	Temporal (OSS self-host), Airflow, Dagster, Prefect	OSS free	Temporal Cloud / managed
Object storage + CDN	AWS S3 + CloudFront, Cloudflare R2, Backblaze B2	Some free egress	Storage + egress
Image transforms	imgproxy (OSS), Cloudinary, Cloudflare Images	OSS free; free tiers	Usage-based
Edge / hosting	Cloudflare, Vercel, Netlify, Fly.io, Render	Generous free tiers	Pro plans
Observability	Sentry, Grafana Cloud, OpenTelemetry (OSS), Better Stack	Free tiers	Usage-based
Product analytics	PostHog, GA4, Umami (OSS), Plausible	Free tiers; OSS free	Usage-based
LLM observability	Langfuse (OSS + cloud)	OSS free; cloud free tier	Usage-based
Email / SMS (OTP)	Resend, SendGrid, AWS SES, MSG91, Twilio	Free tiers (email)	Per email/SMS
Feature flags	Unleash (OSS), Flagsmith, GrowthBook	OSS free; free tiers	Per seat
Uptime monitoring	UptimeRobot, Better Stack	Free tiers	Low monthly

16.6 Recommended stacks by budget

Budget	Stack	Approx monthly cost
Zero-budget student	Ollama or Gemini free tier; pgvector; Postgres FTS; Playwright/Scrapy; Next.js on Vercel free; Supabase/Neon; Upstash; Langfuse OSS; Cloudflare free	Rs.0 - Rs.1,000
MVP / small launch	Paid LLM (routed) + free crawl where possible; managed Postgres + Redis; Cloudflare; Sentry/PostHog free	Rs.5,000 - Rs.20,000

Production	Multi-LLM routing; managed OpenSearch; Temporal Cloud; S3+CloudFront; full observability; autoscaling workers	Rs.50,000+ (LLM + crawl dominated)
------------	---	------------------------------------

SECURITY NOTE

Never ship provider API keys to the browser. All LLM, crawl and affiliate calls go through your backend. Store secrets in the platform secret manager, rotate them, and scope affiliate IDs per merchant.

17. Project Requirements (Hardware, Software, Skills, Budget)

17.1 Hardware

Environment	Minimum	Recommended
Developer laptop	4-core CPU, 8 GB RAM, 20 GB free disk	6-8 core CPU, 16 GB RAM, SSD; GPU optional
Local LLM (optional)	8 GB VRAM or CPU (slow)	12-24 GB VRAM GPU, or use hosted API
Production web	1 vCPU / 2 GB (managed platform)	2-4 vCPU / 8-16 GB, autoscaled
Workers	Shares web initially	Separate autoscaling pool (pipeline bursts)
Database	Managed free tier	Managed Postgres with backups + replica

17.2 Software and accounts

- Software: OS (macOS/Linux/Windows+WSL), Git, Docker Desktop, Node.js 20+, pnpm, Python 3.11+, VS Code, Postman/Insomnia, a DB client (TablePlus/DBeaver), Figma (optional).
- Accounts: GitHub; a hosting/edge provider (Vercel + Cloudflare); a managed Postgres (Supabase/Neon); an LLM provider; a crawl/search provider (or self-host); an observability provider; and affiliate program accounts (Amazon Associates, Flipkart, Cuelinks/EarnKaro).
- Local services: Postgres+pgvector, Redis, OpenSearch (all via docker-compose).

17.3 Skills and roles matrix

Skill	Used for	Level needed
Programming (TS/Python)	Everything	Intermediate+
Data structures & algorithms	Ranking, dedup, search, caching, scheduling	Intermediate
SQL & data modeling	Schema, queries, indexing, analytics	Intermediate
Frontend (React/Next.js)	All user surfaces	Intermediate
Backend / API design	Endpoints, auth, redirect, caching	Intermediate
AI / LLM engineering	Agents, RAG, scoring, evals	Intermediate
Data engineering	Pipelines, connectors, warehousing	Basic-Intermediate
DevOps / cloud	CI/CD, deploy, monitor, scale	Basic
Security & privacy	Secrets, authz, compliance	Basic
Product / UX	Scope, flows, copy, trust	Basic

17.4 Budget tiers

Tier	Who	Monthly	Notes
Tier 0 - Free	Students, hobby	Rs.0 - Rs.1,000	Free tiers + OSS; accept rate limits
Tier 1 - MVP	Solo founder / small team	Rs.5,000 - Rs.20,000	Paid LLM + managed DB; limit crawl
Tier 2 - Production	Launched product	Rs.50,000+	Dominated by LLM tokens + crawling

17.5 Time plan for a solo builder / student (10-12 weeks)

Weeks	Focus	Output
1	Setup, taxonomy, schema	Local stack + seed data
2-3	Connectors, entity resolution, item/list API + UI	Browsable catalogue
4-5	LLM + RAG + Ask Flash	Grounded Q&A with citations
6-7	Agent pipeline, scoring, ranking	First ranked lists
8	Price comparison + affiliate redirect	Monetization path working
9	Search + agent hub + editorial	Full feature surface
10	Testing, evals, performance	Green suite + metrics
11-12	Deploy, monitor, document, demo	Live project + report

17.6 Evaluation mapping (for a DSA / academic project)

Criterion	How this project satisfies it
Correct use of data structures	Hash maps, heaps, tries, graphs, Union-Find, LRU, Bloom filter (Section 15.2)
Algorithmic analysis	Complexity table and trade-off discussion (Section 15.3)
System design	Layered architecture, DFD/ER, sequence diagrams (Sections 8, 10, 13)
Working implementation	Deployed app with real data (Sections 13, 14)
Documentation	This PRD + ADRs + OpenAPI spec
Testing	Unit/contract/E2E/load/AI-eval strategy (Section 13.7)
Originality / impact	AI-agent evidence pipeline with ranking integrity invariant

17.7 Suggested report outline for submission

- Abstract and problem statement (choice overload + review trust).
- Introduction, objectives, scope and limitations.
- Literature/background: e-commerce search, recommender systems, LLMs, RAG.
- System design: architecture, data flow, DFD (0/1), ER diagram, use cases, sequence diagrams.
- Algorithms and data structures: choices, pseudocode and complexity analysis.
- Implementation: tech stack, modules, screenshots, API list.
- Testing and results: test plan, latency/throughput, AI eval metrics.

- Deployment and operations.
- Conclusion, future work and references.

17.8 Likely viva / interview questions

Question	Short answer direction
Why a heap for Top-K instead of sorting everything?	$O(n \log k)$ vs $O(n \log n)$; less memory, faster for small k
How do you merge the same product from multiple stores?	Normalize + Union-Find clustering over blocking/similarity keys
How is search both keyword and semantic?	Hybrid: BM25 inverted index + ANN vector retrieval, then rerank
How do you stop paid placement from changing rank?	Ranking writes restricted to pipeline identity; monetization is read-only on rankings
How do you keep scores reproducible?	Versioned rubrics, fixed prompts, cached evidence, stored score breakdown
How does the pipeline survive failures?	Durable workflow (Temporal) with idempotent, resumable activities
How do you control LLM cost?	Model routing, caching, incremental re-runs, per-run token budgets
How do you prevent hallucinated products?	Answers restricted to retrieved corpus; every claim cites stored evidence

18. Team, Effort & Cost Model

14.1 Indicative team

Role	Count	Focus
Product Manager	1	Scope, rubric, prioritization, integrity policy
Tech Lead / Architect	1	Architecture, infra, code review
Frontend Engineers	2	Next.js app, design system, performance
Backend Engineers	2	Core API, catalogue, offers, redirect
AI/ML Engineers	2	Agents, RAG, scoring, evals, authenticity
Data Engineer	1	Pipelines, warehousing, connectors
Designer	1	UX/UI, brand, editorial layout
QA / SDET	1	Automation, contract tests, release quality
DevOps/SRE	0.5	Cloudflare, CI/CD, observability, cost
Content Editor	1	The Cut editorial quality and citations

14.2 Cost drivers (monthly, illustrative at v1 scale)

Cost centre	Driver	Order of magnitude
LLM inference	Tokens per research run x runs	Largest variable cost; optimize with routing + caching
Crawling/proxies	Requests, anti-bot, transcripts	Second largest; rate-limit aware
Compute	Web + workers + search	Autoscaled; bursty on pipeline runs
Storage/egress	Images, transcripts, artifacts	Image transforms + CDN keep egress predictable
Data services	Postgres, Redis, ClickHouse, OpenSearch	Managed services cost
Observability	Traces, logs, LLM traces	Sample traces to control cost
Affiliate payouts	Not a cost; revenue	Positive contribution per click

UNIT ECONOMICS GUARDRAIL

Track cost-per-ranked-product and gross margin per qualified outbound click. If LLM + crawl cost per published product exceeds affiliate revenue per product over its lifetime, reduce run frequency, cache more, or narrow coverage. This guardrail is a release gate for scaling categories.

19. Risks & Mitigations

Risk	Impact	Likelihood	Mitigation
Scraping/ToS and robots compliance	Legal exposure, blocking	High	Prefer official APIs/feeds; respect robots; rate-limit; legal review; diversify sources
LLM hallucination / wrong scores	Trust damage	Medium	Evidence-first outputs, critic agent, human review, eval harness, no unsourced claims
Review manipulation	Skewed rankings	High	Authenticity classifier, source diversity, discard seeded reviews, periodic re-checks
Affiliate program changes	Revenue risk	Medium	Multi-merchant diversification; direct partnerships; track program terms
Price/availability drift	Bad UX, lost trust	High	Frequent re-crawl, freshness timestamps, verify at redirect, show 'checked X ago'
LLM/crawl cost overrun	Unsustainable economics	Medium	Model routing, caching, incremental runs, budgets, unit-economics gate
Ranking-integrity breach	Existential trust loss	Low	Separate service identities; no paid writes; audit logs; public policy
Category sensitivity (health)	Regulatory/claims risk	Medium	Avoid medical claims; cite sources; disclaimers; expert review
Concentration on few merchants	Dependency	Medium	Onboard more merchants; own audience/email list

20. Compliance, Legal & Trust

- Affiliate disclosure: comply with FTC guidelines and Indian advertising norms (ASCI). Disclose affiliate relationships on item, list and interstitial pages.
- Zero paid ranking: publish a ranking-integrity policy; enforce at the authorization layer so monetization code cannot write ranking data.
- Data protection: comply with India's DPDP Act 2023 and, where applicable, GDPR/CCPA. Consent capture, purpose limitation, data-subject access/erasure, PII minimization.
- Data acquisition: respect robots.txt and site terms; prefer licensed APIs and merchant feeds; maintain a source registry with legal status.
- Intellectual property: product images/descriptions may be third-party; use per merchant/affiliate terms; do not republish copyrighted editorial wholesale.
- Claims: avoid medical/health claims; label scores as opinion derived from cited evidence; include methodology page.
- Accessibility: WCAG 2.1 AA for core flows.
- Security: OWASP ASVS L2, secrets management, least-privilege, audit logging for ranking writes.
- Trademarks: do not imply endorsement by merchants/brands; use nominative fair use.

21. Appendices

Appendix A - Observed flash.co routes & fingerprints

Item	Observation
Routes	/home, /browse, /discovery/{id}, /list/{id}, /item/{id}, /item/{id}/prices, /brand/{slug}, /search?query=, /articles, /articles/{slug}, /articles/section/{section}, /hub, /profile, /how-flash-works, /go?u=
Frontend	Next.js App Router + Turbopack, React Server Components, /_next/static chunks
UI libs	MUI + Emotion + Tailwind utility classes
Fonts	Inter Tight, Playfair Display, Instrument Serif, DM Sans, JetBrains Mono, Bangers, Lilita One
Edge	Cloudflare (server: cloudflare; cf-ray; cf-cache-status; NEL)
Images	i.flash.tech and img.flash.co transform URLs over s3://flash-creatives/... (resize, webp, quality)
Analytics	Google Tag Manager
Affiliate	Amazon tag=flashai-21; Flipkart affid=flashco; /go redirect
Claims	'60+ sources per product', '15,000+ stores', 'top 1%', 'RUN #4,182', 'nothing sponsored'
Merchants seen	Amazon, Flipkart, Myntra, Nykaa, Sweetcare, Kiehls, Plum Goodness, Dot & Key, PharmEasy, Tira and others

Appendix B - Sample normalized records

Product

```
{
  "id": "p_2760",
  "brand_id": "b_formularx",
  "title": "The Formularx Sun Relief Ceramide Silk Sunscreen",
  "category_path": ["Beauty", "Baby & Kids", "Combo"],
  "specs": { "spf": 50, "fragrance_free": true, "skin_types": ["sensitive", "acne-prone"] },
  "images": ["https://img.example.com/a/w:600/h:800/f:webp/..."],
  "aliases": ["B0XXXX", "GTIN-13:8901234567890"]
}
```

Score + Finding

```
{
  "score": { "product_id": "p_2760", "list_id": "1677", "rubric_version": "v3",
    "total": 94, "computed_at": "2026-08-04T00:00:00Z" },
  "findings": [
    { "type": "pro", "text": "Broad-spectrum UV protection verified by multiple sources",
      "evidence_ids": ["e_1", "e_2"] },
    { "type": "con", "text": "Lacks independent SPF lab assay",
      "evidence_ids": ["e_9"] }
  ]
}
```

Appendix C - Glossary

Term	Meaning
Flash Score	0-100 evidence-weighted product score for a given list/rubric
Attribute cluster	A mined, evidence-backed cut of products (e.g. 'no white cast')
RSC	React Server Components
RAG	Retrieval-Augmented Generation
SSE	Server-Sent Events
Entity resolution	Deduplicating the same product across merchants
Rubric version	The weighted scoring definition used at computation time
Provenance	Counts of source types behind a list (expert/YouTube/reviews/Reddit)
Run	A numbered execution of the research pipeline

Appendix D - Source notes & disclaimer

Product analysis in this document is derived from publicly accessible pages of flash.co (home, browse, how-flash-works, hub, articles, one list page and one item page) and the client-side artifacts served to a browser, inspected on October 5, 2026. Technology fingerprints reflect what is observable from outside; internal implementation may differ. The recommended stack, API, data model, pipeline and roadmap are original proposals for building a similar product and are not representations of Flash.co's actual internal systems. This document is not affiliated with, endorsed by, or produced in cooperation with Flash.co.